



01. C++ 언어의 특징과 C에서 C++로

Heesung Oh

<https://github.com/3dapi>





- C++의 주요 특징
- C와 C++의 관계
- C++에서 추가되거나 의미가 달라진 기본형 타입
- C++에서 추가된 편리한 문법과 표준 라이브러리
- C 방식에서 C++ 방식으로의 전환

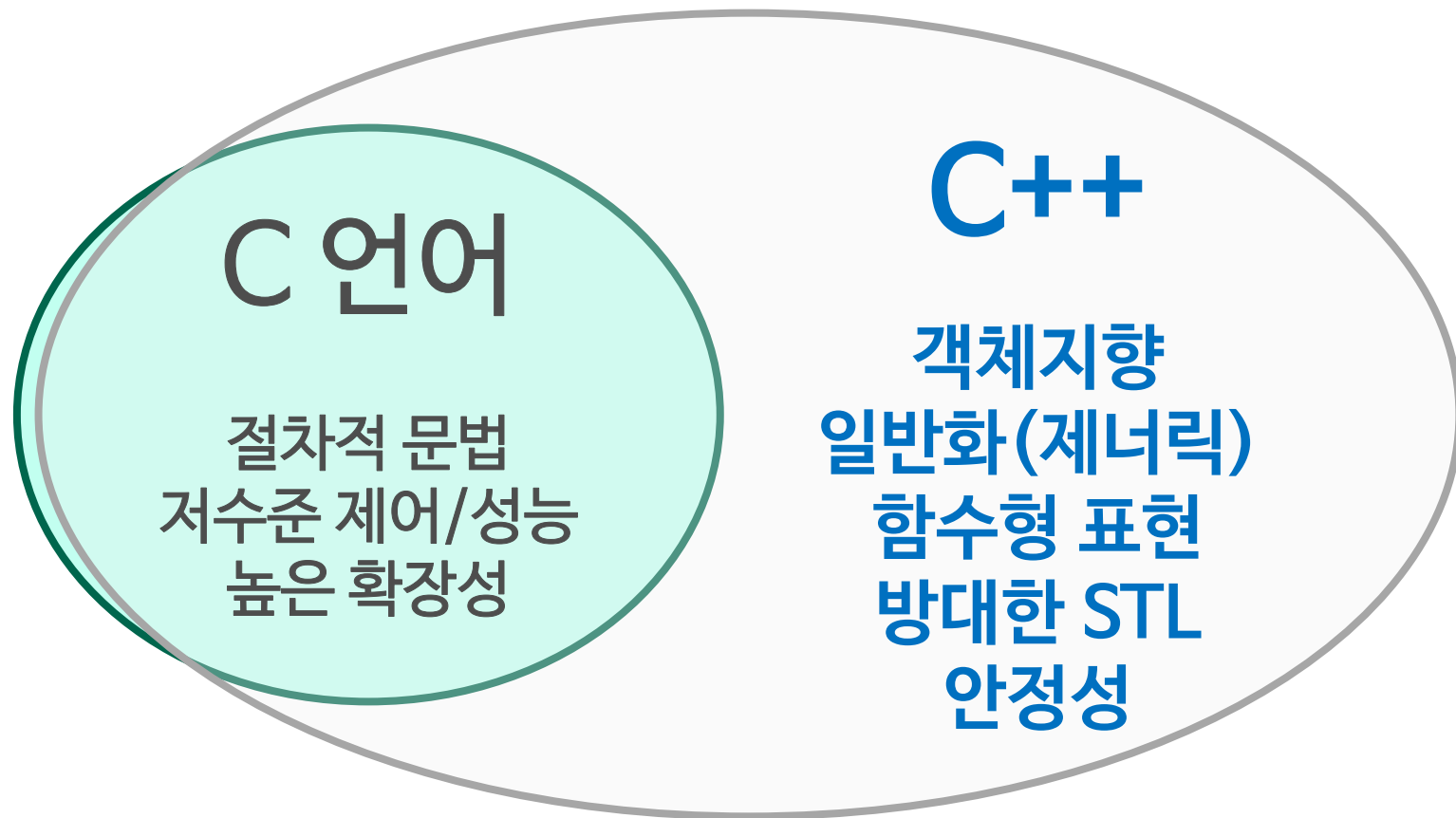




- 절차적 프로그래밍, 객체지향 프로그래밍, 제네릭 프로그래밍 지원
- 문제의 성격에 따라 적절한 방식을 선택하거나 함께 사용
- 프로그램의 모든 부분을 반드시 클래스로 작성할 필요 없음

특징	핵심 내용
절차적 프로그래밍	하드웨어에 가까운 제어를 통한 성능
객체지향 프로그래밍	구조화와 유지보수
표준 라이브러리	자료구조와 알고리즘 재사용
정적 타입과 컴파일 타임 처리	안정성과 성능







- ◆ C의 변수, 조건문, 반복문, 함수, 배열, 구조체, 포인터 활용
- ◆ 작은 함수를 조합하여 처리 과정 표현
- ◆ 간단하거나 성능이 중요한 코드에서 활용
- C++가 높은 성능을 얻을 수 있는 이유
 - ◆ 컴파일된 코드의 네이티브 기계어 실행
 - ◆ 메모리 배치와 객체 수명의 직접 제어
 - ◆ 포인터와 참조를 이용한 데이터 직접 접근
 - ◆ 스택과 동적 메모리의 목적에 따른 선택
 - ◆ 사용하지 않는 기능에 대한 비용 배제
 - ◆ 템플릿 기반 추상화의 컴파일 단계 최적화





● 제로 오버헤드 원칙

- ◆ 사용하지 않는 기능에는 비용이 발생하지 않음
- ◆ 더 안전하고 명확한 표현 사용
- ◆ 불필요한 런타임 비용 최소화

안전하고 명확한 추상화 사용



컴파일 단계에서 코드 최적화



불필요한 실행 시간 비용 최소화





● 포인터와 원소 수를 이용한 배열 전달

```
int SumLowLevel(const int* values, int count)
{
    int result = 0;
    for (int i = 0; i < count; ++i)
    {
        result += values[i];
    }
    return result;
}
```

- 배열의 시작 주소와 원소 수를 별도의 인수로 전달
- 호출하는 쪽에서 두 값의 일치 여부 관리
- 배열 크기를 잘못 전달할 가능성





● std::span을 이용한 배열 전달

```
#include <span>
int Sum(std::span<const int> values)
{
    int result{};
    for (int value : values)
    {
        result += value;
    }
    return result;
}
```

- 시작 주소와 원소 수를 하나의 범위로 표현
- 배열을 전달하면 원소 수 자동 반영
- 원소를 복사하지 않음
- 새로운 저장 공간을 할당하지 않음





● 배열 전달 방식 비교

구분	포인터와 원소 수	<code>std::span</code>
함수 매개변수	포인터와 개수 별도 전달	하나의 범위로 전달
원소 수	호출자가 직접 작성	배열에서 자동 결정
데이터 복사	없음	없음
추가 저장 공간	없음	없음
호출 방법	두 인수의 일치 여부 관리	배열만 전달
오류 가능성	잘못된 원소 수 전달	원소 수 작성 불필요





```
#include <iostream>
#include <span>
int SumLowLevel(const int* values, int count);
int Sum(std::span<const int> values);
int main()
{
    int values[]{10, 20, 30, 40};
    int lowLevelResult{SumLowLevel(values, 4)};
    int spanResult{Sum(values)};

    std::cout << "포인터와 원소 수: "
               << lowLevelResult << '\n';
    std::cout << "std::span: "
               << spanResult << '\n';
}
```

- 두 함수 모두 배열 원소의 합 계산
- Sum()은 배열 크기를 별도의 인수로 작성하지 않음
- 컴파일러가 최적화하면 비슷한 형태의 기계어로 변환 가능





● Character 클래스

```
class Character
{
public:
    void TakeDamage(int damage)
    {
        if (hp -= damage; hp < 0)
            hp = 0;
    }
    int GetHp() const
    {
        return hp;
    }
private:
    int hp = 100;
};
```

- hp: 객체의 상태
- TakeDamage(): 피해 처리 행위
- GetHp(): 현재 체력 조회
- 외부 코드에서 hp 직접 변경 불가
- TakeDamage()를 통해 상태 변경





- 객체지향 프로그래밍의 장점
 - ◆ 데이터 변경 범위 제한
 - ◆ 객체의 유효 상태 유지
 - ◆ 관련 데이터와 기능 통합
 - ◆ 역할과 책임에 따른 구조 분리
 - ◆ 내부 구현 변경의 영향 최소화
 - ◆ 기능별 개발 분담과 협업
- 주의
 - ◆ 클래스를 많이 만드는 것이 목적이 아님
 - ◆ 책임이 지나치게 많은 클래스, 깊은 상속 구조, 불필요한 추상화는 복잡성을 증가
 - ◆ 변경되는 부분을 분리하고 객체 사이의 책임을 명확하게 구성





● C++ 표준 라이브러리

- ◆ 문자열, 자료구조, 파일 입출력, 시간, 난수, 메모리 관리 제공
- ◆ 구현되고 검증된 자료구조와 알고리즘 활동
- ◆ 제작 프로그램의 핵심 기능 집중 → 직접 구현해야 하는 코드 감소

컨테이너	역할
<code>std::array</code>	크기가 고정된 배열
<code>std::vector</code>	크기가 변하는 연속 배열
<code>std::deque</code>	앞과 뒤에서 원소 추가·제거
<code>std::list</code>	연결 리스트
<code>std::map</code>	정렬된 키와 값
<code>std::set</code>	중복되지 않는 정렬된 값
<code>std::unordered_map</code>	해시를 이용한 키와 값
<code>std::string</code>	문자열





● std::vector 와 std::sort

```
#include <algorithm>
#include <iostream>
#include <vector>

int main()
{
    std::vector<int> values { 30, 10, 50, 20, 40 };
    std::sort(values.begin(), values.end());
    for (int value : values)
    {
        std::cout << value << ' ';
    }
    std::cout << value << std::endl;
}
```

- std::vector: 데이터 저장 담당
- std::sort: 정렬 담당
- 동적 배열을 직접 구현할 필요 없음
- 정렬 알고리즘을 직접 구현할 필요 없음





- 변수와 표현식의 타입을 컴파일 시점에 결정
- 서로 대입할 수 없는 타입 검사
- 잘못된 함수 호출 검사
- 조건을 만족하지 않는 템플릿 사용 검사
- 실행 전에 오류 발견
- **타입 추론**으로 컴파일러가 타입을 대신 결정

기능	역할
<code>nullptr</code>	정수값 0과 널 포인터 구분
<code>enum class</code>	열거형 값의 범위 제한
C++ 형 변환 연산자	변환 종류를 명확하게 표현
<code>auto</code>	초기값으로 타입 추론
<code>decltype</code>	표현식으로부터 타입 결정





1.1.4 정적 타입과 컴파일 타임 처리

```
constexpr int Square(int value)
{
    return value * value;
}

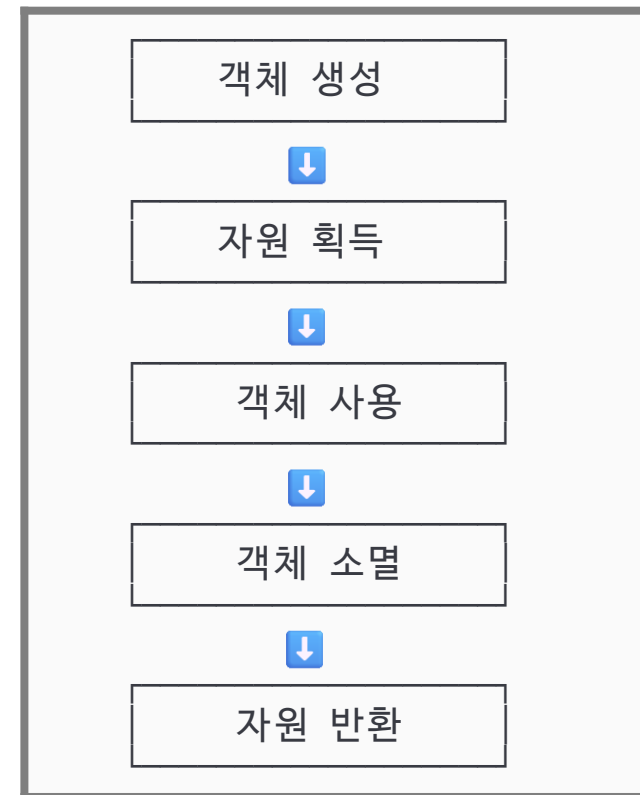
constexpr int size = Square(10);
static_assert(size == 100);
```

- constexpr: 값이나 함수를 상수 표현식으로 사용
- static_assert: 조건이 만족되지 않으면 컴파일 중단
- constexpr 함수는 상황에 따라 컴파일 시점 또는 실행 시점에 계산
- 반드시 컴파일 시점에 평가해야 하는 함수에는 consteval 사용





- 프로그램의 자원:
메모리, 파일, 네트워크 연결, 잠금, 그래픽스 자원
- 자원은 사용이 끝난 시점에 정확하게 반환
- RAI:
객체의 생성과 소멸을 자원 관리에 연결





```
#include <memory>

void ProcessData(bool cancel)
{
    auto values = std::make_unique<int[]>(100);
    values[0] = 10;
    values[1] = 20;

    if (cancel)
    {
        return;
    }

    values[2] = values[0] + values[1];
}
```

- `std::unique_ptr` 객체가 동적 메모리의 소유권 관리
- 함수가 정상 종료되거나 중간에 return되어도 자동 해제
- 예외가 발생해도 객체의 소멸 과정 유지
- `std::vector`, `std::string`, 스마트 포인터도 RAII 기반





- 운영체제와 시스템 소프트웨어
- 게임 엔진과 실시간 콘텐츠
- 그래픽스와 렌더링
- 물리 및 과학 시뮬레이션
- 로봇과 자율주행 시스템
- 임베디드 시스템
- 금융 및 대규모 데이터 처리
- 데이터베이스와 네트워크 서버
- 웹 브라우저와 데스크톱 응용 프로그램
- 개발 도구와 콘텐츠 제작 도구





- ◆ C를 기반으로 발전
- ◆ 변수, 연산자, 제어문, 함수, 배열, 구조체와 포인터를 상당 부분 계승
- ◆ C 학습 경험을 바탕으로 C++ 학습 가능

- C++에서 확장된 기능
 - ◆ 더 엄격한 타입 검사
 - ◆ 참조와 이름 공간
 - ◆ 함수 오버로딩과 기본 인수
 - ◆ 클래스와 객체지향 프로그래밍
 - ◆ 템플릿과 표준 라이브러리
 - ◆ RAII와 스마트 포인터
 - ◆ C++는 C에 클래스만 추가한 언어가 아님





- 입력 변수의 주소 전달
- 변수 타입에 맞는 형식 지정자 작성
- 주소 연산자나 형식 지정자를 잘못 사용하면 예상하지 못한 결과 발생

```
// C 방식
int level = 0;
double speed = 0.0;

scanf("%d %lf", &level, &speed);
printf("Level: %d, Speed: %.1f\n", level, speed);
```





- 변수와 표현식의 타입에 따라 입출력 동작 선택
- 형식 지정자와 주소 연산자를 직접 작성하지 않음

```
// C++ 방식
#include <iostream>

int main()
{
    using std::cin; using std::cout;

    int level = 0;
    double speed = 0.0;

    cout << "레벨과 이동 속도를 입력하세요: ";
    cin >> level >> speed;

    cout << "Level: " << level
         << ", Speed: " << speed << '\n';
}
```





1.2.1 타입을 이용한 안전한 입출력

```
// 공백이 포함된 문자열 입력
#include <iostream>
#include <string>
using namespace std;

int main()
{
    string name;

    cout << "이름을 입력하세요: ";
    getline(cin, name);

    cout << "입력한 이름: " << name << '\n';
}
```

구분	C	C++
입력	scanf()	cin
출력	printf()	cout
형식 지정자	직접 작성	타입에 따라 결정
일반 변수 주소	직접 전달	사용하지 않음
한 줄 문자열	별도 처리	getline()





- 기본형 타입과 변수
- 산술·관계·논리 연산자
- 조건문과 반복문
- 함수
- 배열과 구조체
- 포인터와 함수 포인터
- 동적 메모리
- 파일 입출력

```
struct Item
{
    char name[32];
};

struct Character
{
    int hp;
    int attack;
    struct Item* pItem;
};
```





- C++ 구조체 타입 이름
 - ◆ "struct" 키워드를 반복하지 않아도 됨
 - ◆ C 문법 사용 가능하지만 C++에 적합한 방식으로 확장

```
struct Character
{
    // ...
    Item* pItem;
};
void TakeDamage(Character* character, int damage)
{
    if (character->hp -= damage; character->hp < 0)
    {
        character->hp = 0;
    }
}
```





- 이름 공간
- 참조
- 함수 오버로딩과 기본 인수
- 클래스와 접근 제어
- 생성자와 소멸자
- 상속과 다형성
- 예외 처리
- 템플릿과 템플릿 인수 추론
- RAII와 스마트 포인터
- auto, decltype, nullptr, enum class
- 람다 표현식과 Ranges
- 표준 라이브러리





- C++는 C보다 엄격한 타입 검사
- C의 모든 프로그램을 그대로 받아들이는 완전한 상위 호환 언어는 아님

```
void* memory = nullptr;
```

```
// int* pointer = memory; // C++에서는 오류
```

```
int* pointer = static_cast<int*>(memory)
```





1.2.4 C++의 주요 키워드

구분	주요 키워드
논리값과 널 포인터	bool true false nullptr
이름과 타입 정의	namespace using class typename
객체와 접근 제어	public protected private this friend
상속과 다형성	virtual override final
템플릿과 타입 추론	template auto decltype
형 변환	static_cast const_cast reinterpret_cast dynamic_cast
예외 처리	try catch throw noexcept
컴파일 타임 처리	constexpr consteval constexpr static_assert
메모리와 연산자	new delete operator explicit

```
// int class = 10; // 오류  
// void return(); // 오류
```





- 메모리 누수 / 댕글링 포인터
- 배열 범위를 벗어난 접근
- 여러 함수가 같은 데이터를 변경하는 상태 관리 문제
- 자원의 소유자와 해제 책임 불분명
- 한 부분의 수정이 다른 부분에 미치는 영향
- C++ 사용으로 C의 모든 오류가 자동으로 수정되는 것 아님
- 클래스, RAII, 표준 컨테이너, 스마트 포인터와 타입 시스템으로 자원 관리 범위 축소





- C++의 `std::vector`가 저장 공간과 원소 수를 함께 관리
- 객체의 수명이 끝나면 내부 메모리 자동 해제

```
// C 방식
#include <cstdlib>
void ProcessData()
{
    int* values = static_cast<int*>(std::malloc(sizeof(int) * 100));
    if (values == nullptr) {
        return;
    }
    // 데이터 사용
    std::free(values);
}

// C++ 방식
#include <vector>
void ProcessData()
{
    std::vector<int> values(100);
    // 데이터 사용
}
```





1.3 C++의 기본형 타입

분류	내용
기본형 타입	언어가 직접 제공
복합 타입	포인터, 참조, 배열, 함수 타입
사용자 정의 타입	클래스, 구조체, 공용체, 열거형

기본형 분류	타입
논리형	bool
문자형	char, wchar_t, char8_t, char16_t, char32_t 등
정수형	short, int, long, long long과 unsigned 형식
부동소수점형	float, double, long double
빈 타입	void
널 포인터 타입	std::nullptr_t





- 소문자 l은 숫자 1과 구분하기 어려우므로 대문자 L 권장

접미사	리터럴 타입	예
없음	값을 표현할 수 있는 표준 정수형	100
U	부호 없는 정수형	100U
L	long	100L
LL	long long	100LL
UL	unsigned long	100UL
ULL	unsigned long long	100ULL

```
short smallValue = 10;  
int value = 100;  
long largeValue = 1000L;  
long long veryLargeValue = 10000LL;
```





1.3.2 정수형과 리터럴 접미사

- 2진수 리터럴과 고정 폭 정수형
- C++14부터 **0b**, **0B** 접두사 사용
- 작은따옴표를 자릿수 구분자로 사용
- 정확한 비트 폭이 필요한 경우 `<cstdint>`의 고정 폭 정수형 사용

타입	용도
<code>long long</code>	표준 정수형, 최소 64비트 범위
<code>std::int64_t</code>	정확히 64비트인 정수형
<code>__int64</code>	Microsoft 전용 비표준 타입

```
int flags = 0b1010;  
int decimalValue = 1'000'000;  
  
#include <cstdint>  
std::int32_t id = 100;  
std::uint64_t fileSize = 5000000000ULL;
```





- bool type
 - ◆ 값: true, false

변환	결과
정수 0 → bool	false
0이 아닌 정수 → bool	true
false → 정수	0
true → 정수	1

// 조건의 의미를 직접 표현하기 위해 1대신 true 사용.

```
int hp = 50;
bool isAlive = hp > 0;
while (true)
{
    // 반복 처리
}
```





- bool 값 출력
- cout은 기본적으로 bool 값을 0과 1로 출력
- `std::boolalpha`를 사용하면 `false`, `true`로 출력

```
#include <iostream>
using namespace std;

int main()
{
    bool first = 0;
    bool second = 10;
    bool third = -1;

    cout << std::boolalpha;
    cout << "first  : " << first << '\n';
    cout << "second : " << second << '\n';
    cout << "third  : " << third << '\n';
}
```





1.3.4 확장된 문자형과 리터럴 접두사

표기	타입	용도
'A'	<code>char</code>	일반 문자 코드 단위
<code>L'A'</code>	<code>wchar_t</code>	와이드 문자 코드 단위
<code>u8'A'</code>	<code>char8_t</code>	UTF-8 코드 단위
<code>u'A'</code>	<code>char16_t</code>	UTF-16 코드 단위
<code>U'A'</code>	<code>char32_t</code>	UTF-32 코드 단위

```
char asciiCharacter = 'A';  
wchar_t wideCharacter = L'A';  
char8_t utf8Character = u8'A';  
char16_t utf16Character = u'A';  
char32_t utf32Character = U'A';
```





- C와 동일
- 접미사가 없으면 기본적으로 double
- long double에는 대문자 L 권장
- 일부 실수는 근사값으로 저장

접미사	리터럴 타입	예
없음	double	3.5
f 또는 F	float	3.5f
l 또는 L	long double	3.5L

```
float speed = 3.5f;  
double distance = 1250.75;  
long double extendedPi = 3.1415926535897932384626433832795L;  
double lightSpeed = 2.99792458e8;  
double epsilon = 1.0e-6;
```





● void

- ◆ 값이 없음을 나타내는 타입
- ◆ 반환 타입에 사용하면 반환값이 없는 함수
- ◆ C++에서 매개변수가 없는 함수는 빈 괄호 () 사용
- ◆ void 객체나 배열은 만들 수 없음

```
void PrintMessage()  
{  
    std::cout << "Hello" << '\n';  
}
```

● void*

- ◆ 구체적인 객체 타입이 정해지지 않은 포인터
- ◆ 값을 사용하려면 원래 포인터 타입으로 변환

```
int value = 100;  
void* memory = &value;
```





- NULL Pointer

- ◆ 기존 코드에서는 0이나 NULL 사용
- ◆ C++11부터 널 포인터를 명확하게 표현하는 nullptr 제공
- ◆ 새 C++ 코드에서는 nullptr 사용

```
int* firstPointer = 0;  
int* secondPointer = NULL;  
int* pointer = nullptr;
```

- std::nullptr_t: nullptr의 타입

- ◆ 함수 오버로딩이나 템플릿에서 nullptr 자체를 구분할 때 활용

```
#include <cstddef>  
std::nullptr_t nullValue = nullptr;
```





1.4 C++에서 추가된 편리한 문법과 기능

- 기존 C 문법 사용 가능 → 더 간결하고 명확한 작성 방법 제공
- 프로젝트에서 사용하는 C++ 표준 버전 확인 필요

기능	표준 버전
<code>new, delete, new[], delete[]</code>	C++98
조건에서 변수 선언	C++98
범위 기반 for문 (<code>range for</code>)	C++11
<code><random></code> 난수 라이브러리	C++11
<code>if, switch</code> 초기화 구문	C++17
초기화 구문이 있는 범위 기반 for문	C++20





1.4.1 new / delete 동적 메모리

- 한 개의 인스턴스 → new
- 객체 배열 생성 → new []
- ()를 사용하면 모든 원소를 0으로 값 초기화
- new, delete → C++ 언어의 연산자 (문법으로 지원)
- 할당 실패 시 `std::bad_alloc` 예외가 발생, `try-catch`에서 처리

```
// C방식. cstdlib, cstring 헤더파일 필요
#include <cstdlib>
#include <cstring>
char* memory =
    (char*)(malloc(100 * sizeof(char)));

if (memory != nullptr)
{
    memset(memory, 0, 100);
    free(memory);
}
```

```
// C++
char* memory = new char[100]();

// 메모리 사용

delete[] memory;
```





- new 할당 실패를 **nullptr**로 확인하는 경우

```
#include <new>
char* memory = new (std::nothrow) char[100]();
if (memory != nullptr)
{
    // 메모리 사용
    delete[] memory;
}
```

- 동적 할당과 해제 짝 맞추기
 - ◆ 서로 다른 할당과 해제 방식을 혼합하면 안 됨
 - ◆ 일반적인 새 C++ 코드에서는 **std::vector** 와 스마트 포인터 우선

동적 할당	해제
단일 객체 new	delete
객체 배열 new[]	delete[]
malloc() , calloc() , realloc()	free()





- 조건에서 변수 선언 — C++98

```
if (int value = GetValue())  
{  
    std::cout << value << '\n';  
}  
  
while (Node* node = GetNextNode())  
{  
    node->Process();  
}
```





- 초기화 구문이 있는 if문 — C++17
 - ◆ 선언한 변수의 범위를 제어문 안으로 제한
 - ◆ if, switch는 초기화 구문 지원
 - ◆ while은 초기화 구문을 지원하지 않음

```
if (int value = GetValue(); value > 0)
{
    std::cout << "양수: " << value << '\n';
}
else
{
    std::cout << "0 또는 음수: " << value << '\n';
}
```

```
switch (int command = GetCommand(); command)
{
```





● C++11

```
int values[] = { 10, 20, 30, 40, 50 };  
for (int value : values)  
{  
    std::cout << value << '\n';  
}
```

● C++20

```
for (std::vector<int> values = GetValues(); int value : values)  
{  
    std::cout << value << '\n';  
}
```

형태	용도
일반 for문	인덱스가 필요하거나 일부 원소 순회
범위 기반 for문	모든 원소를 차례로 순회
초기화 구문이 있는 범위 기반 for문	반복에 필요한 범위를 먼저 생성





1.4.4 C++ 난수 라이브러리(C++11)

- C의 rand()와 % 연산보다 난수 생성 역할을 명확하게 분리
- 난수 엔진: 원시 난수열 생성
- 분포: 원하는 범위와 확률 분포로 변환
- 같은 엔진과 같은 Seed → 같은 난수열
- 고정 시드는 오류 재현과 테스트에 활용

```
#include <random>
```

```
std::random_device seed;
```

```
std::mt19937 engine(seed());
```

```
std::uniform_int_distribution<int> dice(1, 6);
```

```
int result = dice(engine);
```

난수 엔진



분포



원하는 범위





- 정렬 범위의 시작 위치와 마지막 원소 다음 위치 전달
- 비교 기준을 지정하지 않으면 오름차순
- `std::greater<>`를 사용하면 내림차순
- `std::span`은 배열을 복사하지 않고 하나의 범위로 표현

```
#include <algorithm>
#include <functional>
#include <span>

int values[] = { 1, 9, 2, 3, 5, 7, 4, 17 };
std::span<int> range(values);

std::sort(range.begin(), range.end());

std::sort(range.begin(), range.end(), std::greater<>());
```





- C++ 특징
- C의 기본 문법을 상당 부분 계승
- cin, cout, getline()을 이용한 타입 기반 입출력
- bool, 확장 문자형, nullptr 등의 기본형과 표현 추가
- new, delete
- 제어문 안의 변수 선언으로 유효 범위 제한
- 범위 기반 for문으로 배열과 컨테이너 순회
- <random>
- std::sort()
- C 방식 → C++ 클래스, RAII, 표준 컨테이너와 타입 시스템 활용





- 자주 사용되는 헤더파일
 - ◆ `iostream` `cstdio` `cstdlib` `cstring`
 - ◆ `algorithm`
 - ◆ `functional`
 - ◆ `string`
 - ◆ `map`
 - ◆ `memory`
 - ◆ `vector`
 - ◆ `cassert`
 - ◆ `unordered_map`
 - ◆ `span`
 - ◆ `random`
 - ◆ `new`

